

1. 回答问题

- 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

有。最近一次的经历是在数学建模比赛中，我们小组三人共同完成建模、代码和论文。当时没有使用 Git，文件交换主要依靠微信群。文件比较小的情况下，可以直接在群内传输；需要共同负责的部分，也可以使用共享文档进行实时编辑。

这种方式确实遇到了一些问题。例如微信无法直接发送文件夹，需要先进行压缩，而我们分别使用 Windows 和 macOS，压缩包在不同系统之间解压时可能会出现文件名乱码，后来只能通过 U 盘直接传递文件。

论文需要经过多轮修改，我们采用了比较直接的版本管理方式：每次进行较大修改时，将整个项目复制一份，并建立“论文 V1”“论文 V2”等文件夹。这样可以保留旧版本，但也产生了大量重复文件。随着版本增加，有时会难以确认某个文件属于哪一版，导致出现难以寻找上一版本文件或遗漏部分文件的情况。

- 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

暂存让开发者在提交前对修改进行筛选和检查。实际开发中，工作区往往会同时包含多个文件、以及多个不同目的的修改，而 `git add` 可以选择其中一部分进入暂存区，再通过 `git diff --cached` 等命令检查即将提交的内容。确认后，`git commit` 才会把这些修改记录成仓库中的一个版本。这样可以使每次提交对应较清晰的逻辑改动，也减少误提交临时文件或无关修改的情况。提交形成的历史记录还可以用于后续比较和版本回退。

- `git branch` 和 `git branch -a` 的区别是什么？查阅资料并回答。

`git branch` 只显示本地分支；而 `git branch -a` 会显示所有分支，包括本地分支和远程跟踪分支。

2. 建立个人仓库并完成commit

- 根据模板仓库建立自己的仓库

 New repository

Search

Repository

+

Settings

Help

Profile

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).

Required fields are marked with an asterisk (*).

Start with a template

Templates pre-configure your repository with files.

ICS-26Fall-FDU/GitLab

Include all branches

If enabled, all branches from the template repository will be included.

Off

1 General

Owner *

Repository name *

xiashinazumi

/

ICS2026

ICS2026 is available.

Great repository names are short and memorable. How about [refactored-octo-happiness?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *

Choose who can see and commit to this repository

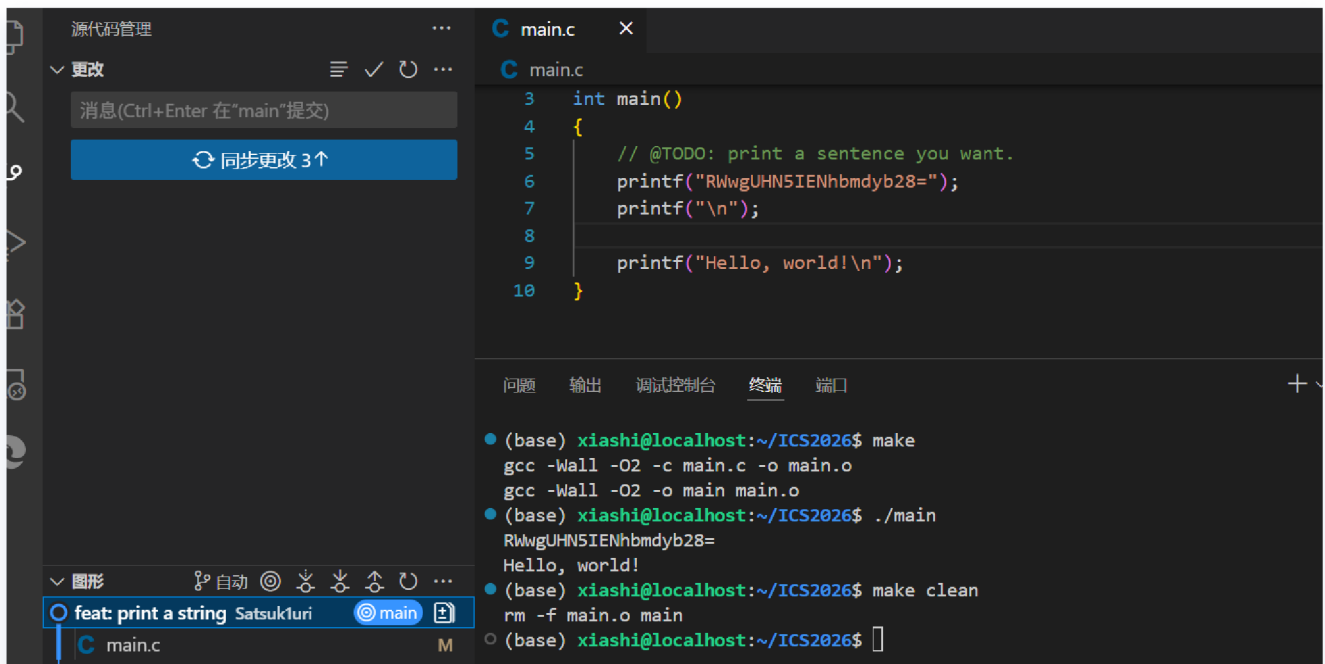
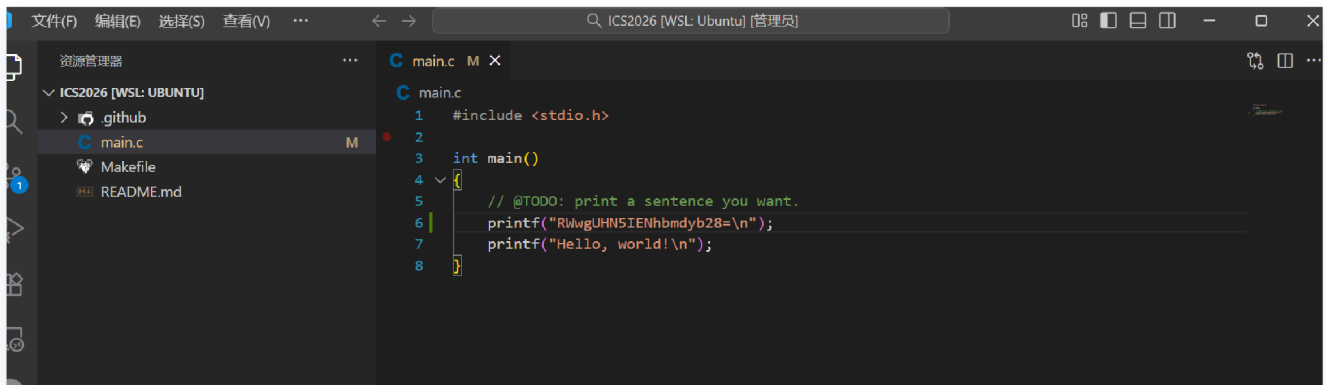
Public

Create repository

- 把创建的仓库clone到本地

```
(base) xiashi@localhost:~$ git clone git@github.com:xiashinazumi/ICS2026.git
Cloning into 'ICS2026'...
Enter passphrase for key '/home/xiashi/.ssh/id_ed25519':
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 8 (delta 0), reused 7 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (8/8), done.
(base) xiashi@localhost:~$
```

- 完成main.c文件中的TODO部分并进行一次commit



3. 阅读网页

1) [commit message规范](#)

主要介绍Git每次提交代码时，需要编写的提交说明（Commit Message）的作用，如：

- 方便快速浏览commit的信息
- 便于快速查找信息
- 便于生成文档（Change log）来观察新旧版本的差异

以及书写规范以及一些建议，包括：

- 包括 **Header**，**Body** 和 **Footer** 三个部分，后面二者可以省略
- 为避免自动换行，任何一行都不得超过72个字符（或100个字符）

- **Header** 只有一行，包括三个字段：**type**（必需，用于说明类别，只能使用规定的7个标识）、**scope**（可选，说明影响范围）和 **subject**（必需，对目的的简短描述，不超过50字符）
- **Body** 是对本次 commit 的详细描述，可以分成多行，和 **subject** 一样应使用第一人称现在时
- **Footer** 用于不兼容变动和关闭Issue
- 对于 **Revert** 特殊情况的说明

除此之外，还介绍了撰写合格Commit message的工具 **Commitizen** 和检查Commit message是否符合格式的 **validate-commit-msg**

2) [Git Flow分支控制](#)

主要介绍了Gitflow的使用规范

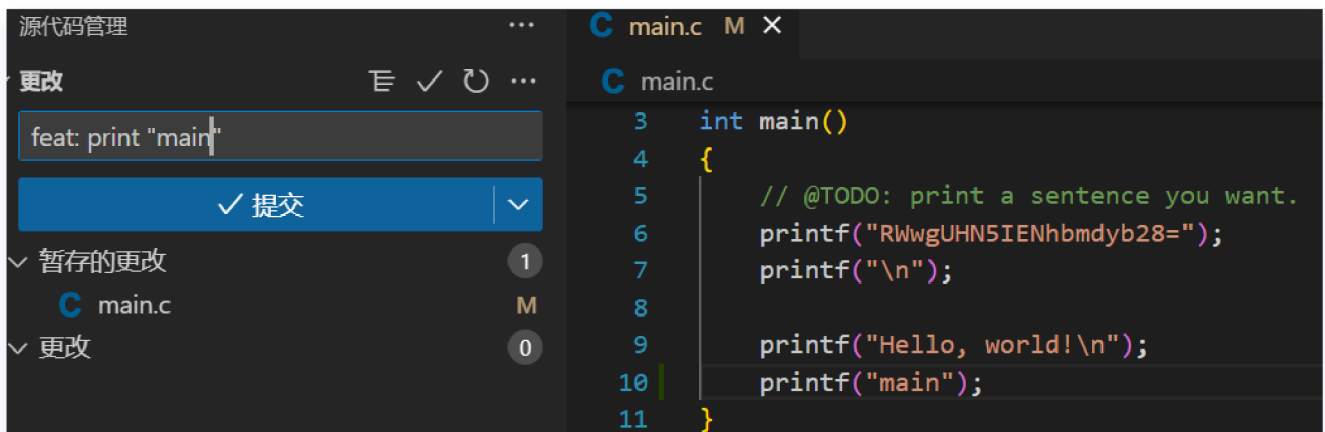
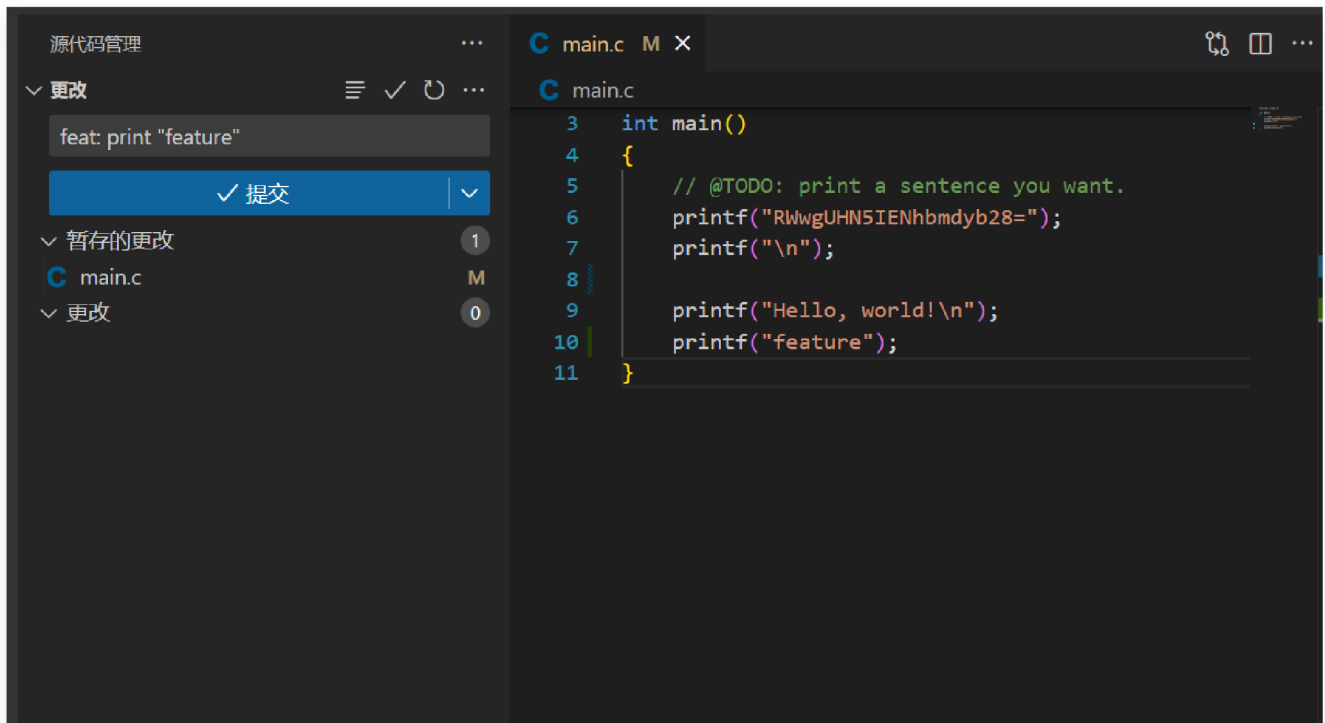
- 关键分支包括master, develop, feature, release, hotfix等
- 不同分支之间的关系，比如develop 分支是基于 master 分支创建，在提交代码后，又重新回到了 master 分支；从develop 分支创建 feature 分支开发新功能，编码自测完成后，再从 feature 分支 push（推送）到 develop 分支；将开发的新代码从 develop 分支推送到 release 分支，而后发布代码 (CI/CD) 到测试环境供测试验证，等等

3) 为什么要学习Git?

学习 Git 能够让开发者建立规范的版本管理和协作开发方式。Git 可以记录代码的修改历史，方便查看差异、恢复旧版本，也能够通过分支和合并支持多人并行开发。与手动管理相比，Git 对修改的记录更清晰，也更适合长期项目和团队协作。

4. 学习Git分支管理

- 新建 **feature** 分支，在该分支以及 **main** 分支上对 **main.c** 分别进行一次修改与提交



- 将 `feature` 分支 merge 到 `main` 分支

出现合并冲突

源代码管理

更改

Merge branch 'feature'

继续

合并更改 1

main.c !

更改 0

图形

feat: print "main" Satsuk1uri

feat: print a string Satsuk1uri

print a string Satsuk1uri

initial commit Satsuk1uri

main.c M

Initial commit Satsuk1uri

origin/main

classroom.yml .github/workflows A

Makefile A

README.md A

main.c A

main.c ! x

main.c

```
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("RWwgUHN5IENhbm dyb28=");
7     printf("\n");
8
9     printf("Hello, world!\n");
10 采用当前更改 | 采用传入的更改 | 保留双方更改 | 比较变更
11 <<<<<< HEAD (当前更改)
12     printf("main");
13 =====
14     printf("feature");
15 >>>>>> feature (传入的更改)
16 }
```

Resolve in Merge Editor

问题 输出 终端

bash - ICS2026

- (base) xiashi@localhost:~/ICS2026\$ make
- gcc -Wall -O2 -c main.c -o main.o
- gcc -Wall -O2 -o main main.o
- (base) xiashi@localhost:~/ICS2026\$./main
- RWwgUHN5IENhbm dyb28=
- Hello, world!
- (base) xiashi@localhost:~/ICS2026\$ make clean
- rm -f main.o main
- (base) xiashi@localhost:~/ICS2026\$

存在合并冲突。请先解决这些问题，然后再提交更改。

来源: Git

显示更改

WSL: Ubuntu

main! 0: 41 0 0 0

行 15, 列 2 空格: 4 UTF-8 LF {} C

在merge editor中解决冲突后，合并成功

The screenshot shows the Visual Studio Code interface with a merge editor open. The top bar indicates the file is 'main.c' and the merge is '正在合并: main.c'. The left sidebar shows the file explorer with 'main.c' selected. The main editor area is split into three panes. The left pane shows the '传入' (Incoming) version of the file, commit 'ad01498' from 'refs/heads/feature'. The right pane shows the '当前' (Current) version, commit 'a9dfa13' from 'refs/heads/main'. Both versions of the code are identical, showing a C program with a 'main' function that prints a long string, 'Hello, world!', and then either 'feature' or 'main'. The bottom pane, titled '结果 main.c', shows the merged result, which is identical to the other two versions but with 'merge' printed instead of 'feature' or 'main'. The status bar at the bottom indicates 'WSL: Ubuntu', 'main!', '0:41', '0:0', '0:0', '行 10, 列 21', '空格: 4', 'LF', and 'C'. A 'Complete Merge' button is visible in the bottom right corner.

```
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("RWwgUHN5IENhbm dyb28=");
7     printf("\n");
8
9     printf("Hello, world!\n");
10    printf("feature");
11 }

2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("RWwgUHN5IENhbm dyb28=");
7     printf("\n");
8
9     printf("Hello, world!\n");
10    printf("main");
11 }

结果 main.c
2
3 int main()
4 {
5     // @TODO: print a sentence you want.
6     printf("RWwgUHN5IENhbm dyb28=");
7     printf("\n");
8
9     printf("Hello, world!\n");
10    printf("merge");
11 }
```

Complete Merge

